MANUAL VISUAL

TESTE DE LOGIN COM SELENIUM WEBDRIVER

Diagramas, fluxo e checklist do exemplo em **test/login.test.js**. Cada página mostra o que o código faz hoje e a forma segura de publicar o mesmo teste.

1 O CAMINHO QUE O EXEMPLO PERCORRE

Do Mocha até a saudação exibida depois do login

1. **Mocha** — Chama o único caso de teste do arquivo.
2. **Chrome** — Builder abre uma sessão do navegador.
3. **Página** — driver.get carrega quick-notes.club.
4. **Formulário** — Preenche e-mail e senha e envia Enter.
5. **Prova** — Lê #user-name e compara o texto.

CAPÍTULO 01

Como o teste funciona

- As peças importadas e o papel de cada uma.
- Os ids da tela de login e da saudação.
- O efeito de cada trecho no navegador.

`Builder` `By.id` `Key.RETURN` `until` `assert`

CAPÍTULO 02

Arrange, Act e Assert

- Preparar o Chrome e abrir a URL.
- Agir como a pessoa que entra na conta.
- Conferir a saudação e encerrar o driver.

`Arrange` `Act` `Assert`

CAPÍTULO 03

Espere o elemento aparecer

- findElement busca uma vez e pode falhar cedo.
- elementLocated repete a busca dentro do prazo.
- O wait devolve o elemento para o assert.

`findElement · 1 vez`

`elementLocated · a cada 200 ms`

CAPÍTULO 04

Credenciais e execução

- Senha lida do ambiente, fora do fonte.
- O script npm test ainda não chama o Mocha.
- Senha já commitada precisa ser trocada no site.

`LOGIN_EMAIL` `LOGIN_PASSWORD` `LOGIN_GREETING`

CAPÍTULO 05

Quando o login falha

- O Mocha corta o teste em 2 segundos.
- Elemento ausente, texto diferente, Chrome preso.
- Ordem para ler a falha e achar a causa.

Mocha padrão — 2 s

describe com timeout — 20 s

CAPÍTULO 06

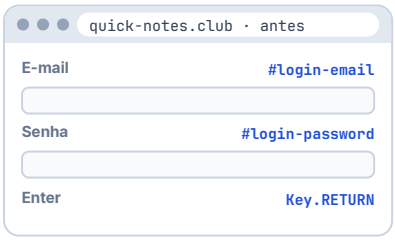
Versão para publicar

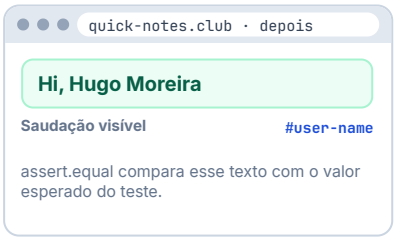
- Timeout de 20s, espera real e finally.
- Variáveis LOGIN_EMAIL e LOGIN_PASSWORD.
- Checklist antes de compartilhar o repositório.

`try { login }` `finally { quit() }`

2 A TELA QUE O TESTE ENXERGA

Três ids. O assert só olha o terceiro.





3 COMO LER CADA CAPÍTULO

O mesmo desenho nas seis páginas

1. O título diz o problema do arquivo de teste.
2. O quadro azul lista o que a página ensina.
3. O cartão vermelho mostra o trecho como está hoje.
4. O cartão verde mostra o trecho para publicar.
5. A tabela liga sintoma, causa e ação.
6. O checklist verde fecha a página com o que conferir.

ARQUIVO	BIBLIOTECA	EXECUTOR
test/login.test.js	selenium-webdriver 4	Mocha 11 · ESM

CAPÍTULO 1

COMO O TESTE DE LOGIN FUNCIONA

Um caso só: abrir o Chrome, entrar em quick-notes.club e confirmar que a saudação da pessoa logada apareceu.

O QUE VOCÊ VAI APRENDER

- Ligar Mocha, Chrome e a página em um fluxo só.
- Ler o papel de Builder, By, Key, until e assert.
- Reconhecer os três ids que o teste procura.
- Associar cada trecho a um efeito no navegador.

1 FLUXO DO CASO DE TESTE

A ordem em que as chamadas acontecem dentro do it

1 build()
Cria um ThenableWebDriver para o Chrome. Os comandos seguintes entram na fila da sessão.

→

2 get(url)
Navega até https://quick-notes.club/ e espera o carregamento da página.

→

3 sendKeys
Digita no #login-email e, no #login-password, digita a senha e Key.RETURN.

→

4 wait + assert
Espera #user-name ficar visível, lê o texto e compara com a saudação esperada.

→

5 quit()
Fecha o navegador. No arquivo atual isso só roda se o assert passar.

2 PEÇAS IMPORTADAS

Cada nome no topo do arquivo tem um trabalho

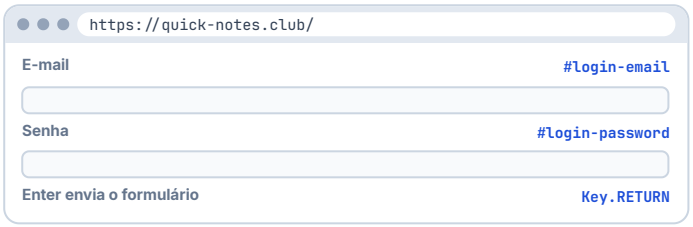
`assert`	Compara o texto lido com o texto esperado e falha o teste se forem diferentes.
`Builder`	Monta a sessão. forBrowser(Browser.CHROME) escolhe o Chrome instalado na máquina.
`By.id`	Vira o CSS *[id="..."]. É o localizador dos campos e da saudação.
`Key.RETURN`	É a tecla Enter. O exemplo usa Enter para enviar o formulário.
`until`	Guarda condições de espera: elementIsVisible e, na versão corrigida, elementLocated.
`describe / it`	São do Mocha. O arquivo descreve um módulo e um caso: login com credenciais válidas.

ORDEM NO ARQUIVO

1. Imports de assert, Builder, By, Key e until.
2. describe nomeia o módulo de login.
3. it executa Arrange, Act e Assert.
4. quit() fecha o Chrome no caminho que passa.

3 IDS QUE A PÁGINA PRECISA TER

O teste quebra se um desses ids mudar



Hi, Hugo Moreira · texto de #user-name

SE O ID MUDAR

#login-email some: a falha acontece antes de digitar.

#login-password some: o e-mail já foi digitado e o envio não ocorre.

#user-name some: o formulário foi enviado e a prova do login não aparece.

4 O QUE CADA TRECHO PRODUZ

Leia o arquivo de cima para baixo

Trecho	Efeito	Se faltar
`Builder + get`	Abre o Chrome já na tela de login.	O teste para antes de achar qualquer campo.
`sendKeys nos dois ids`	Preenche o formulário e dispara o envio.	NoSuchElementError no campo ausente.
`wait + getText + assert.equal`	Confirma que a sessão mostrou a saudação certa.	Timeout ou falha de igualdade. O capítulo 3 detalha a espera.

CHECKLIST DE LEITURA

- Sei apontar quem abre o Chrome e quem fecha.
- Sei os três ids: e-mail, senha e saudação.
- Sei que Enter está no sendKeys da senha.
- Sei qual texto o assert compara.

5 APLIQUE NESTE ARQUIVO

Três marcações antes de mudar qualquer linha

1 Abra o teste
O caso está em test/login.test.js. Há um describe e um único it.

2 Marque o fluxo
Circule build, os dois sendKeys e o assert.equal. Essa é a história do login.

3 Confira os ids
Na página, confirme #login-email, #login-password e #user-name.

CAPÍTULO 2

ARRANGE, ACT E ASSERT

O exemplo já separa o teste em três comentários. Cada fase tem um motivo para existir e um tipo de falha próprio.

O QUE VOCÊ VAI APRENDER

- O que entra em Arrange, Act e Assert neste login.
- Por que o assert olha a tela, e não o código do site.
- Qual fase explica cada tipo de falha.
- Onde o quit() precisa viver para sempre rodar.

ARRANGE

Prepara

Cria o driver do Chrome e abre a URL. No fim desta fase a tela de login está no navegador e nenhum campo foi preenchido.

build() devolve um ThenableWebDriver: dá para chamar get() sem await no build. Await no build deixa explícito o momento em que a sessão falhou ao abrir.

PRODUZ

Chrome aberto em quick-notes.club, com o formulário ainda vazio.

ACT

Age

Localiza #login-email e #login-password, digita e manda Enter. É o gesto da pessoa usuária. O teste ainda não decide se o login deu certo.

sendKeys aceita texto e teclas na mesma chamada. A senha e Key.RETURN vão juntos no segundo campo.

PRODUZ

O formulário foi enviado. O teste ainda não sabe se a conta entrou.

ASSERT

Confere

Espera a saudação, lê #user-name e compara com o texto esperado. assert.equal falha com a diferença exata entre o recebido e o esperado.

quit() fecha o Chrome. No arquivo atual ele está depois do assert, então uma falha pula o fechamento.

PRODUZ

Teste verde quando o texto de #user-name é o esperado. Teste vermelho com os dois textos na mensagem.

4 O MESMO TESTE, COM AS FASES VISÍVEIS

Valores sensíveis ficam em variáveis. A estrutura é a do arquivo.

test/login.test.js — fases

```
// Arrange — navegador na tela de login
const driver = await new Builder().forBrowser(Browser.CHROME).build();
await driver.get(url);

// Act — a pessoa preenche e envia
await driver.findElement(By.id('login-email')).sendKeys(email);
await driver.findElement(By.id('login-password')).sendKeys(senha, Key.RETURN);

// Assert — a sessão mostra a saudação esperada
const nome = await driver.wait(until.elementLocated(By.id('user-name')), 8000);
await driver.wait(until.elementIsVisible(nome), 8000);
assert.equal(await nome.getText(), saudacaoEsperada);
```

Arrange

Abre a sessão e a URL. Para aqui se o Chrome não subir.

Act

Digita nos dois ids e envia Enter. Para aqui se um campo sumiu.

Assert

Lê a saudação. Para aqui se o texto não for o esperado.

quit()

No arquivo atual só roda depois do assert. O capítulo 6 muda isso.

5 QUAL FASE EXPLICA A FALHA

Comece pela mensagem e volte uma fase

Mensagem	Fase	O que verificar
Session not created, Chrome ausente	Arrange	Chrome instalado e selenium-webdriver 4 resolvendo o driver.
NoSuchElementError em login-email ou login-password	Act	A URL abriu a tela certa e os ids continuam iguais.
Timeout ou textos diferentes no assert	Assert	O login concluiu e o texto de #user-name é o combinado.

CHECKLIST DAS TRÊS FASES

- Arrange só prepara. Não digita senha.
- Act só executa o gesto do usuário.
- Assert compara um resultado visível.
- quit() fica numa fase que sempre roda.

6 APLIQUE NAS TRÊS FASES

O comentário já existe. A responsabilidade de cada fase precisa continuar separada.

1. **Arrange**
   Só Chrome e URL. Nenhum sendKeys neste bloco.
2. **Act**
   Só o preenchimento e o Enter. O teste ainda não decide o resultado.
3. **Assert**
   Só a leitura de #user-name e o assert.equal. Anote qual fase a falha cita.

CAPÍTULO 3

ESPERE O ELEMENTO APARECER

A espera do arquivo pede 5 segundos, mas a busca do #user-name acontece uma vez, fora do ciclo. Se o login ainda não renderizou, o prazo não chega a ser usado.

O QUE VOCÊ VAI APRENDER

- A diferença entre achar agora e esperar aparecer.
- Por que elementIsVisible sozinho não repete a busca.
- Como elementLocated consulta de novo a cada 200 ms.
- Usar o elemento que o wait devolve.

1 BUSCA ÚNICA, ESPERA DEPOIS

É o que test/login.test.js faz hoje

espera atual — falha cedo

```
await driver.wait(
  until.elementIsVisible(
    driver.findElement(By.id('user-name'))
  ),
  5000
);
const saudacao = await driver
  .findElement(By.id('user-name'))
  .getText();
```

findElement já dispara quando a condição é criada. Se #user-name ainda não está no DOM, a promise rejeita. driver.wait, ao receber essa rejeição, encerra na hora. Os 5000 ms só valem quando o elemento existe e isDisplayed() responde false.

2 BUSCA REPETIDA DENTRO DO PRAZO

elementLocated usa findElements e tenta de novo

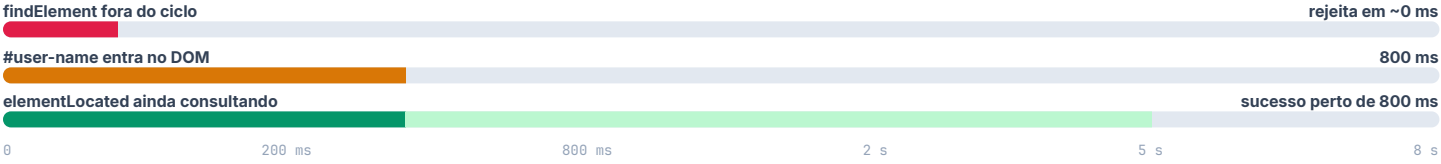
espera corrigida — dentro do prazo

```
const nome = await driver.wait(
  until.elementLocated(By.id('user-name')),
  8000
);
await driver.wait(
  until.elementIsVisible(nome),
  8000
);
const saudacao = await nome.getText();
```

elementLocated chama findElements. Lista vazia é resultado falso, então o wait tenta outra vez a cada 200 ms. Quando o id entra na página, a condição devolve o WebElement. A segunda espera confirma que ele está visível. getText usa esse mesmo objeto.

3 LINHA DO TEMPO: O LOGIN RENDERIZA AOS 800 MS

A busca única já falhou. A busca repetida ainda está no prazo.



A faixa verde-clara é o resto da janela de 8 segundos, que fica sem uso quando o elemento aparece cedo. A faixa vermelha é a busca que não volta a tentar.

0 ms

findElement dispara uma vez. Se o id não existe, a promise rejeita.

200 ms

elementLocated consulta de novo. Lista vazia faz o wait continuar.

800 ms

O id entrou na página. A condição devolve o WebElement.

Depois

elementIsVisible confirma a tela. getText usa esse elemento.

4 QUAL ESPERA USAR

As duas condições respondem perguntas diferentes

Pergunta	Condição	Resultado enquanto espera
O id já está no DOM?	`until.elementLocated(By.id(...))`	Repete a busca. Devolve o elemento quando achar.
Esse elemento está visível?	`until.elementIsVisible(elemento)`	Chama isDisplayed() no elemento que você já tem.
As duas coisas, em ordem	`located, depois visible`	Cobre redirect lento e elemento presente porém oculto.

CHECKLIST DA ESPERA

- A localização está dentro do wait.
- A visibilidade usa o elemento achado.
- O assert lê esse elemento, sem buscar de novo.
- O prazo é maior que o login real da página.

5 TROQUE A ESPERA DO ARQUIVO

A busca da saudação passa a viver dentro do prazo

1 Localize
until.elementLocated(By.id('user-name')) no primeiro wait.

2 Mostre
until.elementIsVisible no elemento que o primeiro wait devolveu.

3 Leia
getText nesse mesmo elemento. O prazo de 8000 ms cobre o render do login.

CAPÍTULO 4

CREDENCIAIS E COMO EXECUTAR

O teste atual escreve e-mail e senha dentro de sendKeys. Quem clona o repositório lê a senha. O script npm test também ainda não executa o Mocha.

O QUE VOCÊ VAI APRENDER

- Tirar e-mail e senha do arquivo de teste.
- Falhar com uma mensagem clara se o ambiente estiver vazio.
- Fazer npm test rodar o Mocha neste arquivo.
- Trocar a senha que já entrou no histórico do Git.

1 SEGREDO DENTRO DO FONTE

O arquivo de hoje segue este formato

sendKeys — no repositório

```
driver.findElement(By.id('login-email'))
  .sendKeys('usuario@email.com');
driver.findElement(By.id('login-password'))
  .sendKeys('senha-em-texto-puro', Key.RETURN);
```

Os valores reais estão no teste commitado. Este quadro mostra só o formato. Apagar a linha numa correção futura tira o segredo dos commits novos. Os commits antigos continuam com a senha. Troque a senha dessa conta no site.

2 SEGREDO NO AMBIENTE

O processo lê, o Git não guarda

ambiente — fora do Git

```
const EMAIL = process.env.LOGIN_EMAIL;
const SENHA = process.env.LOGIN_PASSWORD;
if (!EMAIL || !SENHA) {
  throw new Error(
    'Defina LOGIN_EMAIL e LOGIN_PASSWORD.'
  );
}
```

Sem as variáveis, o teste para com uma frase que diz o que falta. A saudação esperada pode ir em LOGIN_GREETING, porque também é dado do ambiente e não do código.

3 VARIÁVEIS DESTE EXEMPLO

Exporte no terminal antes do npm test

APP_URL	Endereço aberto no Arrange. Padrão útil: https://quick-notes.club/
LOGIN_EMAIL	Valor digitado em #login-email. Obrigatório.
LOGIN_PASSWORD	Valor digitado em #login-password. Obrigatório.
LOGIN_GREETING	Texto esperado de #user-name. No exemplo atual: Hi, Hugo Moreira.

NO TERMINAL

```
export LOGIN_EMAIL=...
export LOGIN_PASSWORD=...
export LOGIN_GREETING='Hi, Hugo Moreira'
npm test
```

4 NPM TEST AINDA NÃO RODA O CASO

O Mocha está nas dependências, o script aponta para outro comando

package.json — script

```
"test": "mocha test/login.test.js --timeout 20000"
```

O package.json deste repositório declara mocha e "type": "module", então o teste em ESM é compatível com o Mocha 11. O script test precisa chamar esse arquivo. Sem isso, npm test termina sem abrir o Chrome.

O QUE O SCRIPT ATUAL FAZ

Imprime uma mensagem de erro e sai com código 1.

Não chama o Mocha e não abre o Chrome.

A troca é apontar "test" para test/login.test.js.

5 TABELA DE SOLUÇÕES RÁPIDAS

Segredo e comando de execução

Situação	Causa	Ação
A senha aparece no Git	sendKeys recebeu um texto literal e isso foi commitado.	Passe a ler o ambiente e troque a senha da conta no site.
npm test não abre o Chrome	O script test não chama o Mocha.	Aponte o script para test/login.test.js.
Erro pedindo LOGIN_EMAIL	O terminal subiu sem as variáveis.	Exporte e-mail, senha e saudação e rode de novo.

CHECKLIST ANTES DE COMPARTILHAR

- Nenhum sendKeys leva senha literal.
- A senha antiga da conta foi trocada.
- npm test chama o Mocha neste arquivo.
- O comando exporta as três variáveis.

6 TIRE O SEGREDO E LIGUE O COMANDO

O Git fica com o fluxo. O terminal fica com a senha.

1. **Apague o literal**
   sendKeys passa a receber LOGIN_EMAIL e LOGIN_PASSWORD.
2. **Troque a senha**
   A senha que já foi commitada continua no histórico. Troque-a no site.
3. **Aponte o npm test**
   O script test chama mocha test/login.test.js --timeout 20000.

CAPÍTULO 5

QUANDO O TESTE DE LOGIN FALHA

Quatro falhas cobrem quase todo erro deste arquivo: o Mocha desiste em 2 segundos, o elemento não está lá, o texto mudou, ou o Chrome fica aberto depois do erro.

O QUE VOCÊ VAI APRENDER

- Separar o prazo do Mocha do prazo do Selenium.
- Ler NoSuchElementError e TimeoutError.
- Usar a diferença do assert para achar texto trocado.
- Fechar o navegador também no caminho de erro.

1. O Mocha corta aos 2 segundos

O prazo padrão do Mocha é 2000 ms. Abrir o Chrome e esperar a saudação passa disso. O it deste arquivo é uma arrow function, então this.timeout() dentro dele não altera o caso. O prazo vale no describe, que já é uma function comum.

```
Timeout of 2000ms exceeded
```

AÇÃO

this.timeout(20000) no describe. O it pode continuar como function, para o this do Mocha existir.

2. O campo ou a saudação não existe

findElement rejeita assim que a busca única falha. By.id vira CSS. Confira se a URL é a tela de login e se o id continua no HTML.

```
no such element: *[id="user-name"]
```

AÇÃO

Troque a busca da saudação por elementLocated. Confira os ids do formulário no HTML da página.

3. A saudação veio diferente

O elemento apareceu, o texto não é o esperado. assert.equal mostra os dois lados. Atualize LOGIN_GREETING ou corrija a conta usada no teste.

```
Expected values to be strictly equal
```

AÇÃO

Leia o texto que a página mostrou e alinhe LOGIN_GREETING a ele.

4. O Chrome continua aberto

quit() está depois do assert. Qualquer throw anterior pula essa linha e deixa o processo do navegador vivo. O finally do capítulo 6 roda nos dois caminhos.

```
processo chrome ainda listado
```

AÇÃO

Feche o Chrome que sobrou e rode de novo com quit() dentro de finally.

5 DOIS RELÓGIOS AO MESMO TEMPO

O Selenium pode pedir 5 segundos. O Mocha encerra o teste aos 2.



A barra azul só teria chance de terminar se o Mocha esperasse o conjunto inteiro. this.timeout(20000) no describe cobre sessão, página e as duas esperas.

6 ORDEM PARA DIAGNOSTICAR

Pare na primeira linha que explica o sintoma

Passo	Olhe	Conclusão prática
1	A primeira linha vermelha do Mocha	2000 ms aponta para o prazo do runner, antes de culpar o login.
2	O id citado no NoSuchElementError	O id diz se a falha foi o formulário ou a saudação.
3	Os dois textos do assert	A página logou. O texto esperado é que precisa ser alinhado.
4	Se o Chrome ficou aberto	Faltou finally. Feche o processo e rode o teste outra vez limpo.

CHECKLIST DE UMA FALHA

- O prazo do Mocha é maior que o do wait.
- A mensagem cita o id que faltou.
- O assert mostra os dois textos.
- O Chrome fecha mesmo com erro.

7 LEIA A PRIMEIRA LINHA E PARE

Uma mensagem basta para escolher o conserto

1

2000 ms

É o Mocha. Coloque this.timeout(20000) no describe.

2

Um id na mensagem

A busca não achou o campo ou a saudação. Vá para o capítulo 3.

3

Dois textos no assert

O login aconteceu. Alinhe LOGIN_GREETING ao texto da tela.

CAPÍTULO 6

A VERSÃO PRONTA PARA PUBLICAR

O mesmo login, com sessão aguardada, espera que repete a busca, segredos no ambiente e navegador fechado nos dois caminhos.

O QUE VOCÊ VAI APRENDER

- Juntar as correções num único caso de teste.
- Onde cada mudança entra no arquivo.
- O comando que executa com as variáveis.
- O checklist final antes do push.

test/login.test.js — publicável

```
import assert from 'node:assert/strict';
import { Builder, Browser, By, Key, until } from 'selenium-webdriver';

const URL = process.env.APP_URL ?? 'https://quick-notes.club/';
const EMAIL = process.env.LOGIN_EMAIL;
const SENHA = process.env.LOGIN_PASSWORD;
const SAUDACAO = process.env.LOGIN_GREETING;

describe('Testes do Módulo de Login', function () {
  this.timeout(20000);

  it('permite o login com credenciais válidas', async function () {
    if (!EMAIL || !SENHA || !SAUDACAO) {
      throw new Error('Defina LOGIN_EMAIL, LOGIN_PASSWORD e LOGIN_GREETING.');
    }
    const driver = await new Builder().forBrowser(Browser.CHROME).build();
    try {
      await driver.get(URL);
      await driver.findElement(By.id('login-email')).sendKeys(EMAIL);
      await driver.findElement(By.id('login-password')).sendKeys(SENHA, Key.RETURN);
      const nome = await driver.wait(until.elementLocated(By.id('user-name')), 8000);
      await driver.wait(until.elementIsVisible(nome), 8000);
      assert.equal(await nome.getText(), SAUDACAO);
    } finally {
      await driver.quit();
    }
  });
});
```

Ambiente
E-mail, senha e saudação ficam fora do arquivo.

20 s
O describe espera a sessão, a página e as duas buscas.

Espera
Localiza #user-name e só então testa se está visível.

finally
quit() roda com o teste verde ou vermelho.

1 O QUE MUDOU

Seis trocas em relação ao arquivo atual

- E-mail, senha e saudação vêm do ambiente.
- O describe espera 20 segundos.
- build() é aguardado. A falha de sessão aparece ali.
- elementLocated repete a busca de #user-name.
- elementIsVisible usa o elemento já encontrado.
- finally chama quit() se o assert passar ou falhar.

COMANDO

Exporte LOGIN_EMAIL, LOGIN_PASSWORD e LOGIN_GREETING no terminal. Com o script do capítulo 4, npm test abre o Chrome, faz o login e fecha o navegador.

HISTÓRICO

Publicar este arquivo novo não apaga a senha dos commits antigos. Troque a senha da conta no quick-notes.club.

2 SCRIPT E DADOS

Dois arquivos, responsabilidades separadas

Lugar	Conteúdo
`package.json`	`"test": "mocha test/login.test.js --timeout 20000"`
Ambiente do terminal	APP_URL, LOGIN_EMAIL, LOGIN_PASSWORD, LOGIN_GREETING=Hi, Hugo Moreira
`test/login.test.js`	Só o fluxo. Nenhum segredo. Chrome instalado na máquina que roda o teste.

CHECKLIST ANTES DE PUBLICAR

- O título diz que o login válido funciona.
- Arrange, Act e Assert continuam separados.
- A espera relocaliza #user-name.
- Senha, timeout, finally e npm test conferidos.

3 PUBLIQUE NESTA ORDEM

Arquivo, script e uma execução com o Chrome fechando sozinho

1. **Substitua o teste**
   O it desta página entra no lugar do caso atual. Sem senha no fonte.
2. **Ajuste o package.json**
   npm test precisa abrir este arquivo pelo Mocha, com 20 segundos.
3. **Rode uma vez**
   Exporte as três variáveis. O teste verde termina com o Chrome fechado.